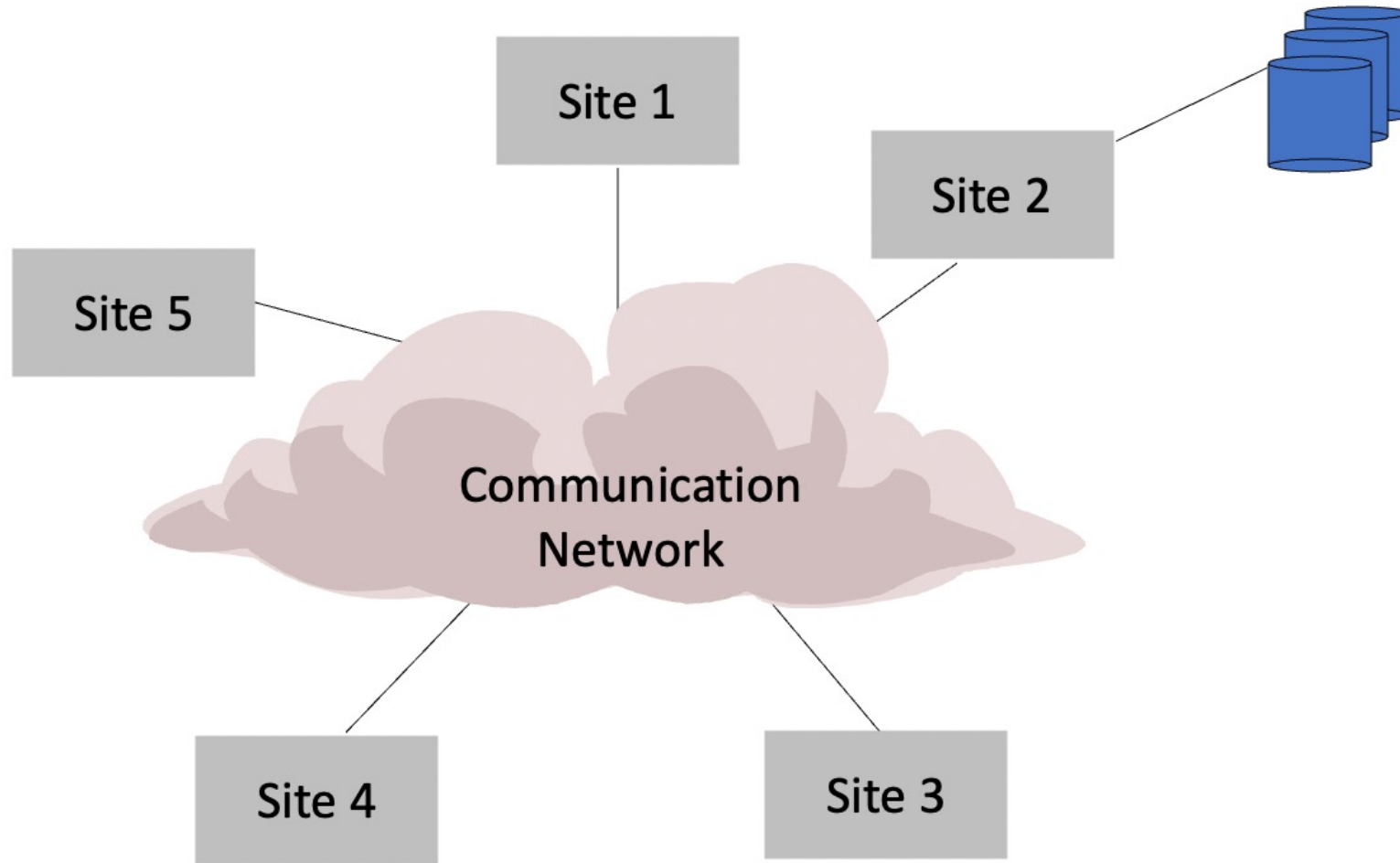


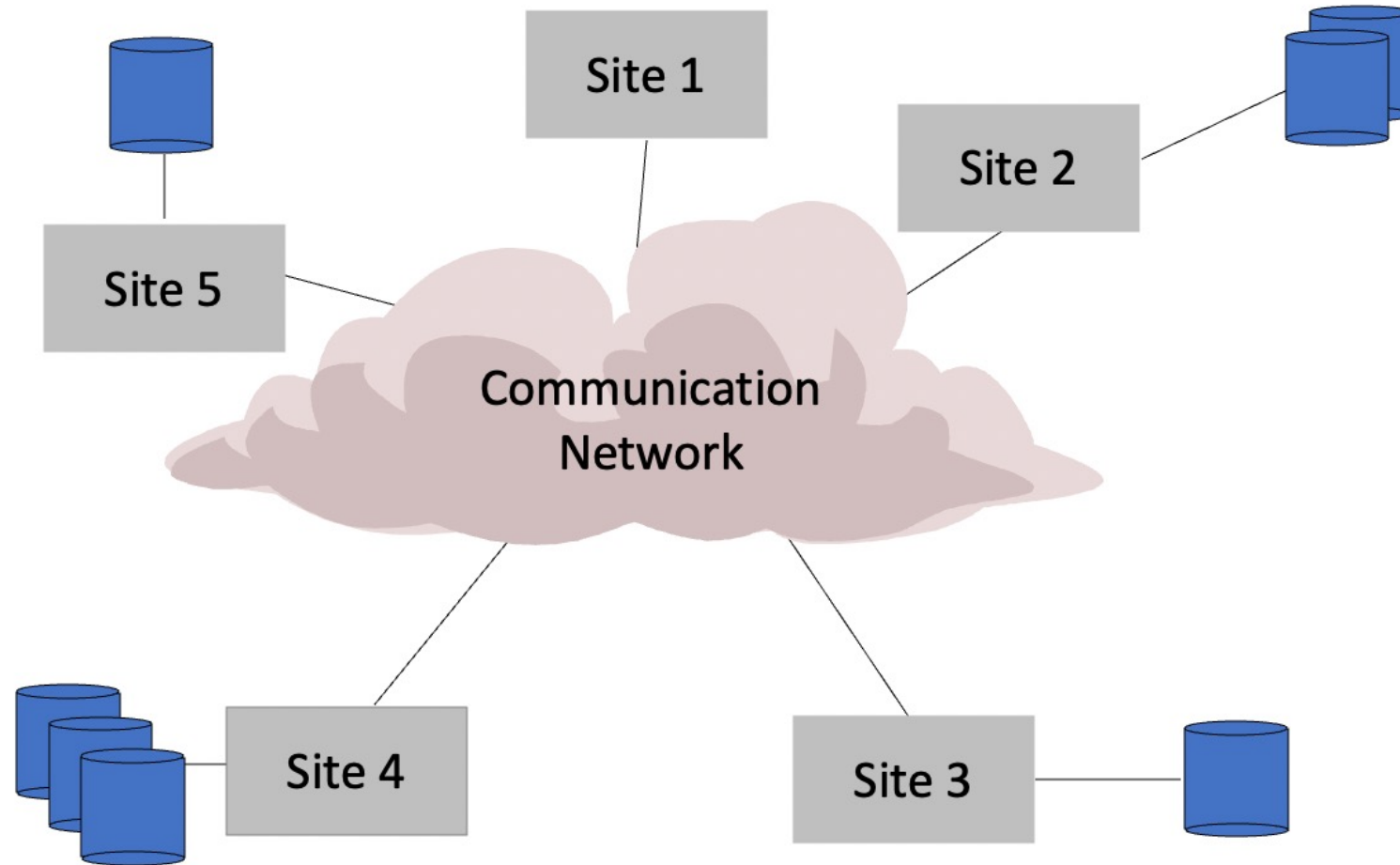
Cloud Based Data Engineering

Lecture 5

Centralized DBMS



Distributed DBMS



GFS

GFS – Google File System

Google File System (GFS) is a distributed file system developed by Google.

- A scalable distributed file system for large distributed data intensive applications
- Designed for system-to-system interaction, and **not for** user-to-system interaction.

The logo for The Google File System, featuring the word "The" in blue and red, "Google" in its multi-colored font, "File" in blue and red, and "System" in yellow and blue.

GFS – Key Observations

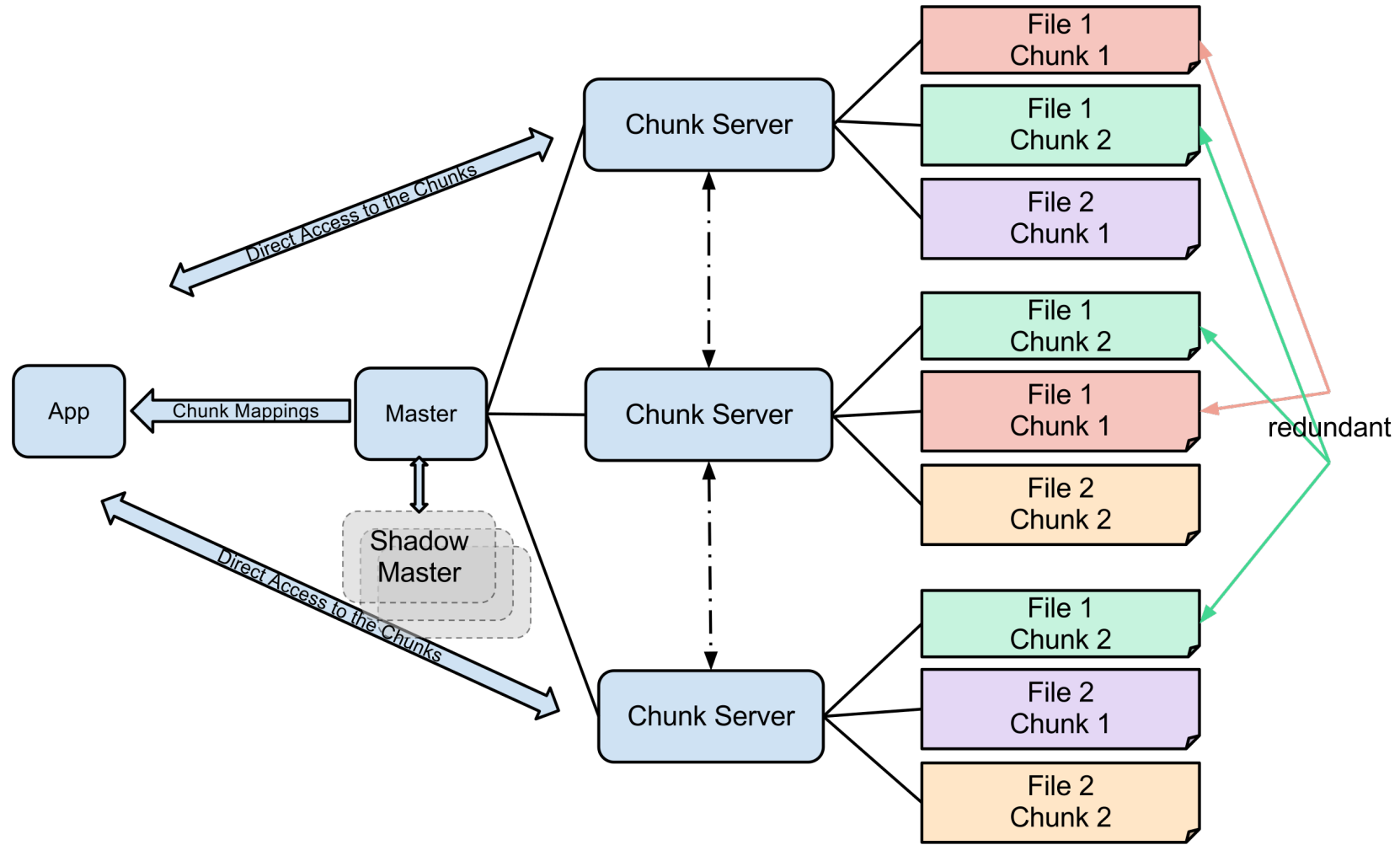
Component failures are the norm - constant monitoring, error detection, fault tolerance

Huge files - Multi GB files are common I/O operations

Files are mutated by appending new data - focus of performance optimization

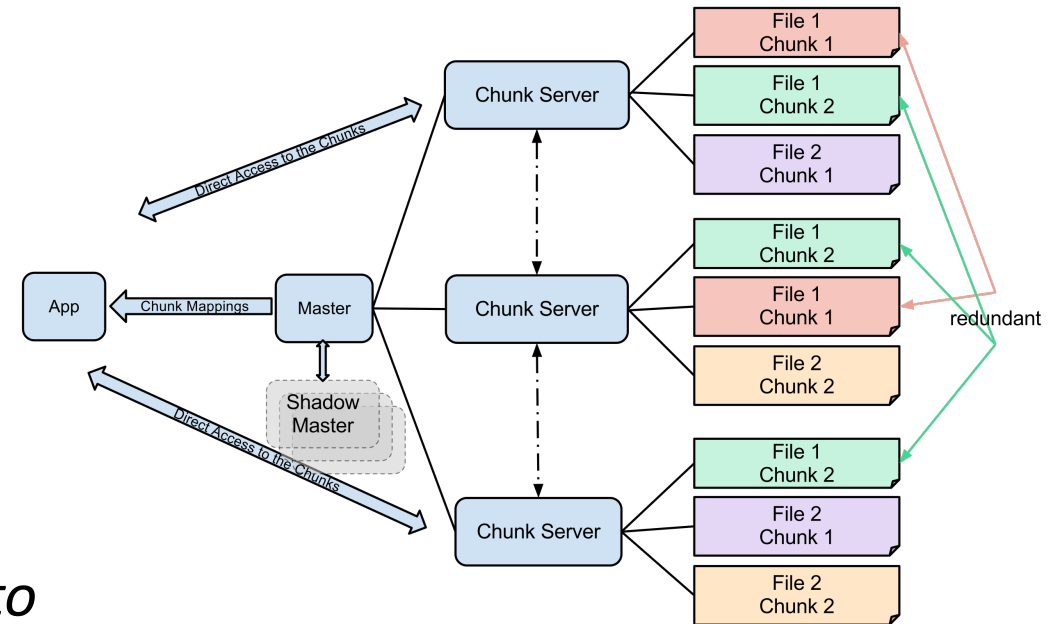
Co-designing the applications and APIs benefits overall system by increasing flexibility

GFS Architecture



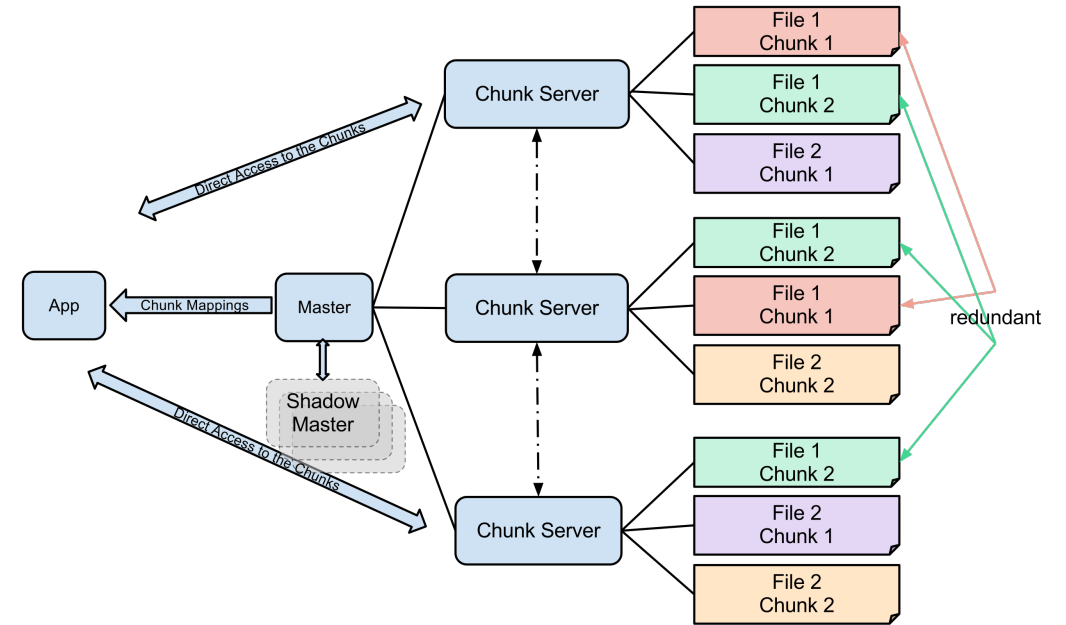
The Master

- Communicates with chunk-servers
- Give instructions
- Maintains all file system metadata
- Helps make sophisticated chunk placement and replication decision
- *For reading and writing, client contacts Master to get chunk locations, then deals directly with chunkservers*



Chunkservers

- Files are broken into chunks.
- Chunk size is 64 MB
- Each chunk is replicated on 3 (default) servers



Yarn

What is YARN?

- Resource Manager takes programming
- Makes it easy to interact to another application Hbase, Spark etc.
- Different Yarn applications can co-exist on the same cluster so MapReduce, Hbase, Spark all can run at the same time bringing great benefits for manageability and cluster utilization.

Components of YARN

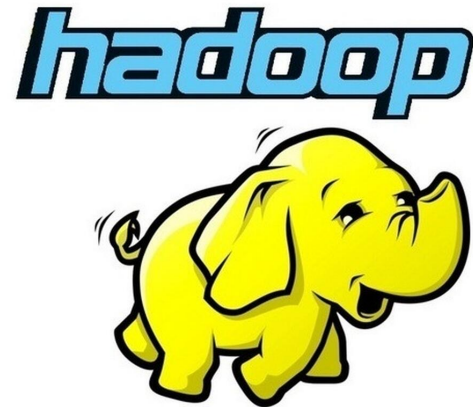
1. **Client:** submitting MapReduce jobs.
2. **Resource Manager:** manage the use of resources across the cluster
3. **Node Manager:** launching and monitoring the computer containers on machines in the cluster.
4. **Map Reduce Application Master:** checks tasks running the MapReduce job.

The application master and the MapReduce tasks run in containers that are scheduled by the resource manager.

Hadoop

HADOOP

- **Hadoop** runs applications using the MapReduce algorithm, where the data is processed in parallel with others.
- **Hadoop** is used to develop applications that could perform complete statistical analysis on huge amounts of data.



HADOOP

- Allows distributed processing of large datasets across clusters of computers
- Hadoop is designed to scale up from **single server** to **thousands of machines**
- It provides a distributed file system (**HDFS**) that stores data on the compute nodes, providing very high aggregate bandwidth across the cluster.

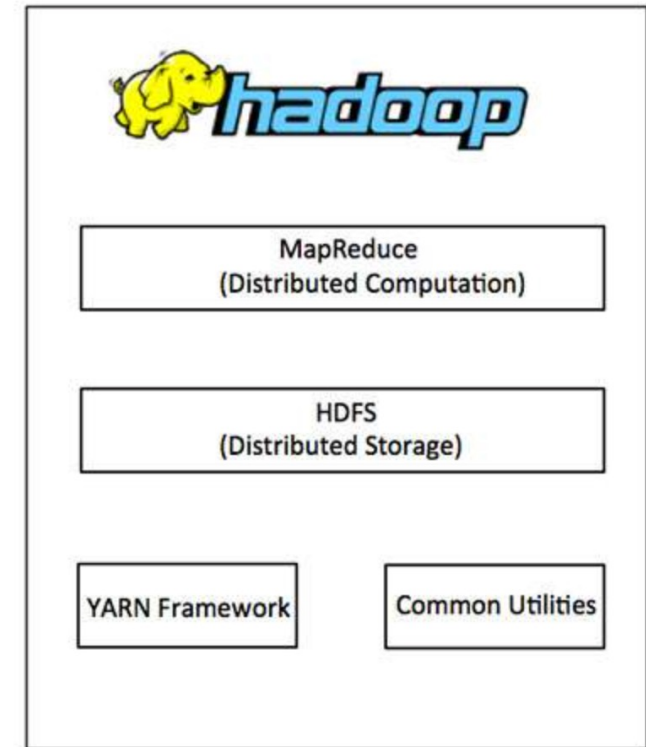
Hadoop - Why ?

- Need to process huge datasets on large clusters of computers
- Very expensive to build reliability into each application
- Nodes fail every day
 - Failure is expected, rather than exceptional
 - The number of nodes in a cluster is not constant
- Need a common infrastructure
 - Efficient, reliable, easy to use
 - Open Source, Apache Licence

HADOOP Architecture

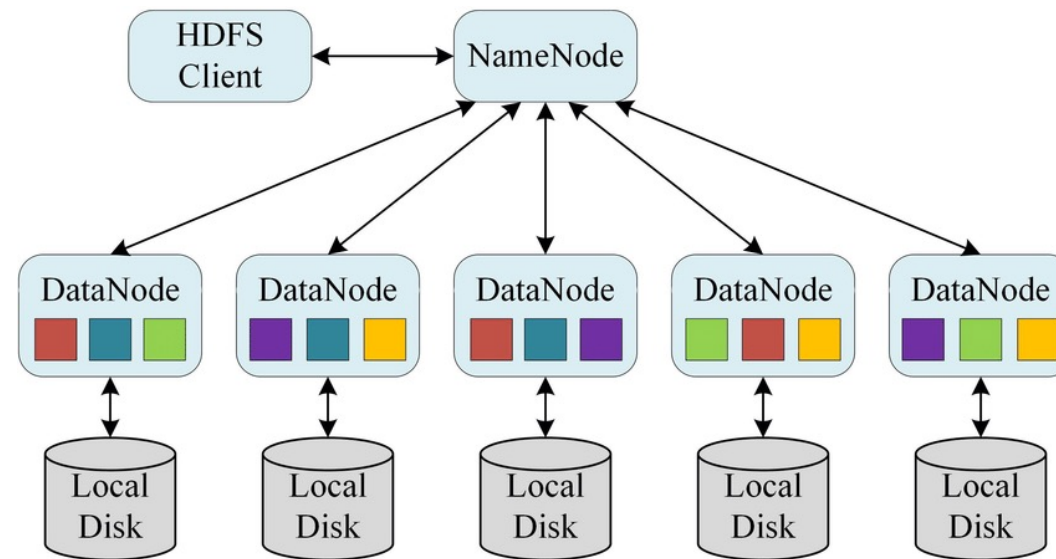
Hadoop has two major layers:

1. Processing/Computation layer (MapReduce)
2. Storage layer (Hadoop Distributed File System - HDFS)



HDFS

The Hadoop Distributed File System (HDFS) is based on the Google File System (GFS) and provides a distributed file system that is designed to run on commodity hardware. It has many similarities with existing distributed file systems.



HDFS

The differences from other distributed file systems are significant. It is highly fault-tolerant and is designed to be deployed on low-cost hardware. It is suitable for applications having large datasets.

Hadoop Modules

Hadoop Common – These are Java libraries and utilities required by other Hadoop modules.

Hadoop YARN – This is a framework for job scheduling and cluster resource management.

Where to use HDFS?

Very Large Files

Files should be of hundreds of megabytes, gigabytes or more.

Streaming Data Access

The time to read whole data set is more important than latency in reading the first.

HDFS is built on write-once and read-many-times pattern.

Commodity Hardware

It works on low-cost hardware.

Where not to use HDFS?

Low Latency data access

Applications that require very less time to access the first data should not use HDFS

Lots Of Small Files

Multiple Writes

It should not be used when we have to write multiple times

Advantages

1. Allows the user to quickly write and test distributed systems.

Efficient and automatic, utilizes the underlying parallelism of the CPU cores

2. Does not rely on hardware to provide fault-tolerance and high availability (FTHA)

Hadoop library itself has been designed to detect and handle failures at the application layer.

3. Servers can be added or removed from the cluster dynamically and Hadoop continues to operate without interruption.

4. **It is compatible on all the platforms since it is Java based.**

How Does It Work?

1. Data is initially divided into directories and files.
2. Files are divided into sized blocks of 128M and 64M (preferably 128M).
3. These files are then distributed across various cluster nodes for further processing.
4. HDFS, being on top of the local file system, supervises the processing.
5. Checking that the code was executed successfully.
6. Performing the sort that takes place between the map and reduce stages.
7. Sending the sorted data to a certain computer.
8. Writing the debugging logs for each job.

Hadoop – Linear Scalability

1. If you have **twice** as many computers, jobs run **twice** faster
2. If you have **twice** as much data, jobs run **twice** slower
3. If you have **twice** as many computers, you can store **twice** as much data!

